

LAPORAN PRAKTIKUM
SISTEM OPERASI
MODUL VI
PROSES SEKUENSIAL DAN KONKUREN



Disusun Oleh:

GHAZA ZIDANE NURRAIHAN

2311104038

SE-07-01

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2025

BAB I

TUJUAN

A. Tujuan

1. Mahasiswa mampu membuat proses berjalan secara sekuensial.
2. Mahasiswa mampu membuat proses berjalan secara konkuren.

Catatan:

1. Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir.
2. Untuk soal source code, kumpulkan SS-nya saja.
3. Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh:
root@username.
4. Berikan identitas nama - nim dalam bentuk comment di Source Code.
5. Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!

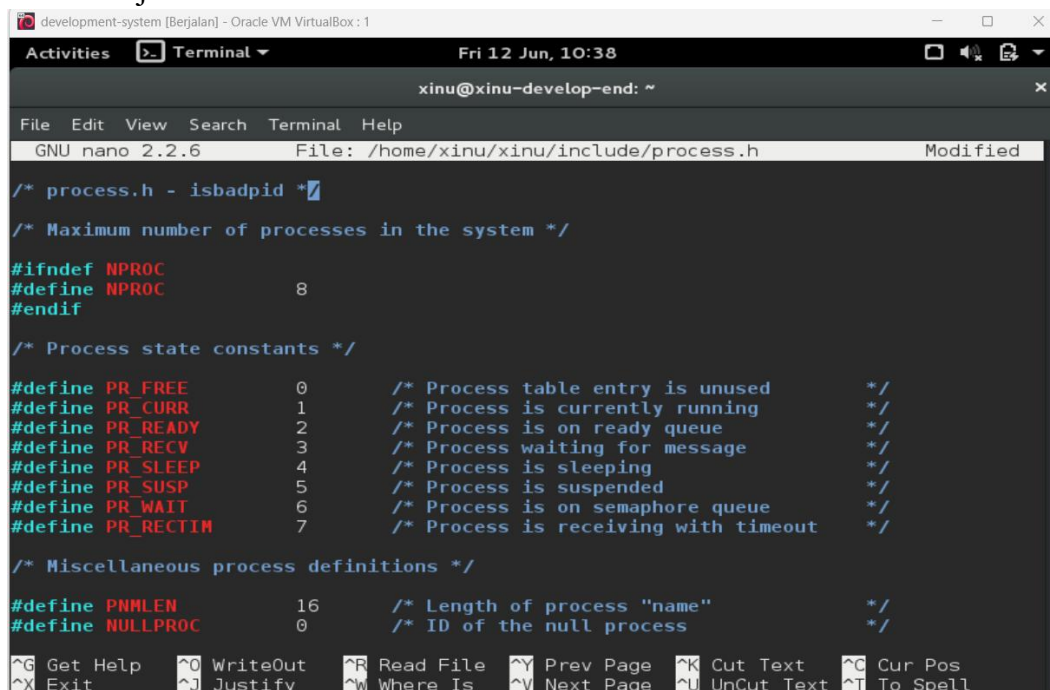
BAB II

JURNAL

1. **[10 Poin]** Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!

Ubah menjadi 8



```
development-system [Berjalan] - Oracle VM VirtualBox: 1
Activities Terminal Fri 12 Jun, 10:38
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: /home/xinu/xinu/include/process.h Modified

/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC 8
#endif

/* Process state constants */

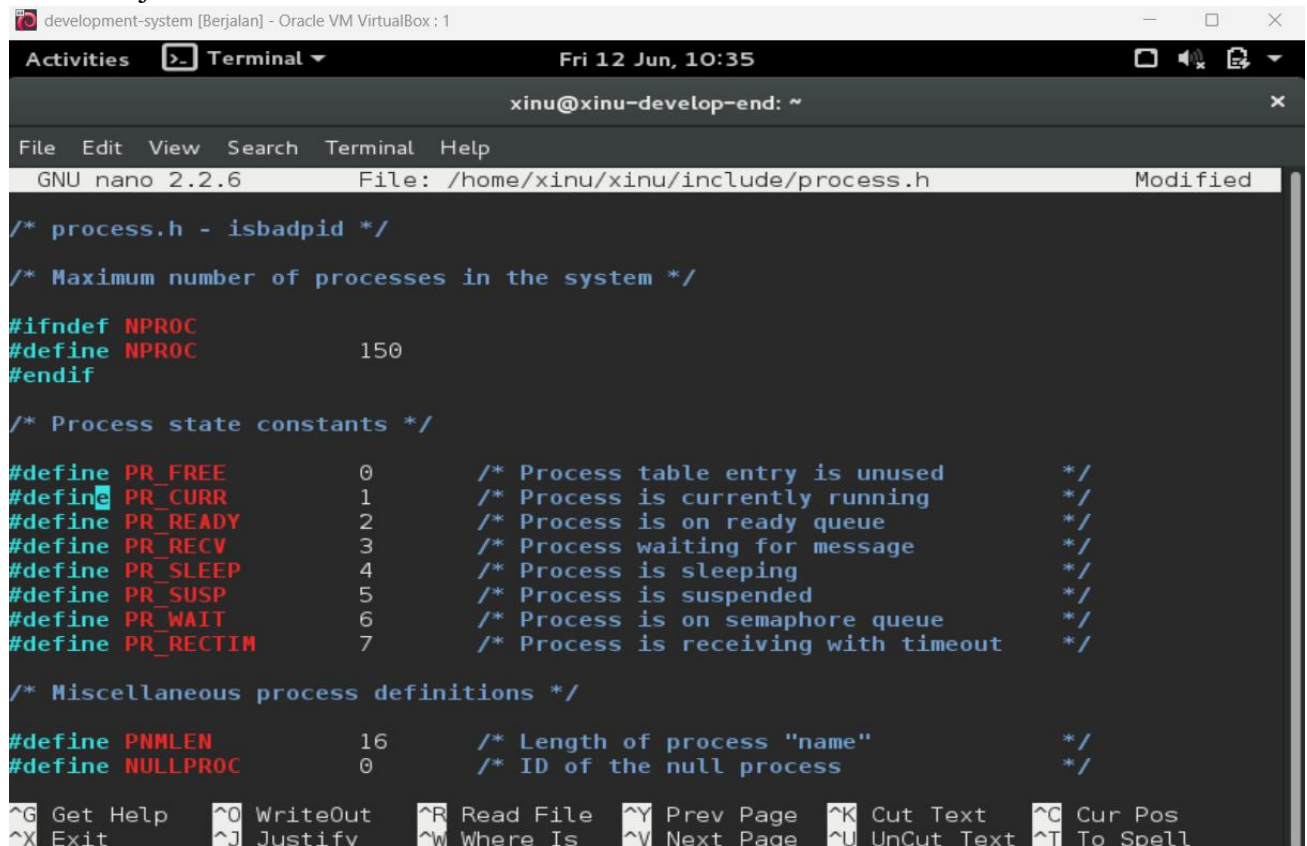
#define PR_FREE 0 /* Process table entry is unused */
#define PR_CURR 1 /* Process is currently running */
#define PR_READY 2 /* Process is on ready queue */
#define PR_RECV 3 /* Process waiting for message */
#define PR_SLEEP 4 /* Process is sleeping */
#define PR_SUSP 5 /* Process is suspended */
#define PR_WAIT 6 /* Process is on semaphore queue */
#define PR_RECTIM 7 /* Process is receiving with timeout */

/* Miscellaneous process definitions */

#define PNMLEN 16 /* Length of process "name" */
#define NULLPROC 0 /* ID of the null process */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

Ubah menjadi 150 =



```
development-system [Berjalan] - Oracle VM VirtualBox : 1
Activities Terminal Fri 12 Jun, 10:35
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: /home/xinu/xinu/include/process.h Modified

/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC 150
#endif

/* Process state constants */

#define PR_FREE 0 /* Process table entry is unused */
#define PR_CURR 1 /* Process is currently running */
#define PR_READY 2 /* Process is on ready queue */
#define PR_RECV 3 /* Process waiting for message */
#define PR_SLEEP 4 /* Process is sleeping */
#define PR_SUSP 5 /* Process is suspended */
#define PR_WAIT 6 /* Process is on semaphore queue */
#define PR_RECTIM 7 /* Process is receiving with timeout */

/* Miscellaneous process definitions */

#define PNMLEN 16 /* Length of process "name" */
#define NULLPROC 0 /* ID of the null process */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

2. 20 Poin] Jalankan kode sekuensial!

Jawab =

Source Code

```
/* main.c - main */
#include <xinu.h>

void A(){
    kprintf("Proses A jalan\n");
}
void B(){
    kprintf("Proses B jalan\n");
}

process main (void)
{
    kprintf("== SEKUESIAL ==\n");
    A();
    B();
    return OK;
}
```

Output code :

```
Obtained IP address 10.0.3.15 (0x0a00030f)
== SEKUESIAL ==
Proses A jalan
Proses B jalan
```

3. [20 Poin] Jalankan kode konkuren!

Jawab =

Source Code =

```
process A(){
    kprintf("Proses A jalan\n");
    return OK;
}
process B(){
    kprintf("Proses B jalan\n");
    return OK;
}
process main (void)
{
    kprintf("=== Konkuren ===\n");
    resume(create(A, 1624, 20, "A", 0));
    resume(create(B, 1624, 20, "B", 0));
    sleeps(500);
    return OK;
}
```

Output code:

```
Obtained IP address 10.0.3.15 (0x0a00030f)
=== Konkuren ===
Proses A jalan
Proses B jalan
```

4. [50 Poin] Buatlah 2 **proses** produser dan konsumen. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```
int32 n = 0; //global variabel

void produser(void){ int32
    i;
    for (i=1; i<=1000; i++){ n++;
    }
}

void konsumer(void){ int32
    i;
    for (i=1; i<=1000; i++){
        printf("Nilai dari n adalah %d\n",n);  }
}
```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal). Jelaskan mengapa hasilnya seperti itu

Jawab :

1. Hasil Pengamatan Program

Ketika program dijalankan di OS Xinu, output yang dihasilkan oleh proses konsumer tidak berurutan secara rapi (acak-acakan). Terjadi fenomena di mana angka 0 atau angka yang sama dicetak berulang kali, terjadi lompatan angka yang drastis, dan nilai akhir n saat dicetak tidak sinkron dengan jumlah baris eksekusi.

2. Analisis Penyebab (Mengapa Hasilnya Mengejutkan)

Hasil tersebut tidak sesuai dengan intuisi awal karena adanya 3 fenomena utama dalam Sistem Operasi, yaitu:

- A. Terjadinya Race Condition (Kondisi Balapan) Variabel int32 n merupakan variabel global (shared memory) yang diakses dan dimodifikasi secara konkuren (bersamaan) oleh proses produser dan konsumer. Karena tidak ada mekanisme sinkronisasi (seperti Semaphore atau Mutex), kedua proses ini saling berebutan untuk membaca dan menulis nilai n.
- B. Operasi n++ Tidak Bersifat Atomik Di dalam bahasa pemrograman tingkat tinggi, perintah n++ terlihat hanya seperti satu langkah. Namun, di tingkat instruksi mesin (Assembly/CPU), operasi ini dipecah menjadi 3 tahapan:
 1. Read: Mengambil nilai n dari RAM ke Register CPU.
 2. Increment: Menambah nilai di Register dengan 1.
 3. Write: Menyimpan kembali nilai baru dari Register ke RAM.

Karena OS menggunakan Preemptive Scheduling, CPU bisa menghentikan (preempt)

proses produser secara mendadak di tengah-tengah ketiga langkah tersebut sebelum sempat melakukan tahap Write. Akibatnya, nilai baru hilang (Lost Update) karena proses lain membaca data yang sudah kedaluwarsa.

- C. Penjadwalan CPU yang Non-Deterministic Urutan eksekusi proses sepenuhnya diatur oleh CPU Scheduler internal OS Xinu. Kita tidak bisa memprediksi kapan CPU akan memberikan jatah waktu (time slice) kepada produser dan kapan kepada konsumen. Jika konsumen mendapat jatah waktu berturut-turut sebelum produser sempat menambah nilai, maka konsumen akan terus mencetak nilai n yang lama.

3. Kesimpulan

Program ini menunjukkan bukti nyata dari bahaya interferensi antar-proses yang tidak disinkronisasi. Solusi untuk masa depan agar hasilnya sesuai intuisi adalah dengan menerapkan Mutual Exclusion (pemastian bahwa hanya satu proses yang boleh memodifikasi variabel n dalam satu waktu).

Jawab =

REFERENSI

<https://en.wikipedia.org/wiki/Xinu>